# Explanation 1: Setup, Configuration & DEM File Parsing

**Covers:** Lines 1–171 of `code_edited.py`

---

## 1.1 — What Is This Pipeline?

This code takes **CS2 demo files** ( `.dem` ) — raw binary recordings of professional matches — and classifies each player into a tactical role (Entry, Support, Lurker, AWPer, IGL, Rifler) using **unsupervised machine learning**.

The pipeline has two phases:

1. **Feature Engineering (G1):** Parse demos → extract 14 behavioral metrics per player per round
2. **Clustering (G2):** Group similar behaviors → label each group as a role

---

## 1.2 — Library Imports: Why Each One Matters

```
import subprocess, sys, os, glob, warnings, traceback, time
from pathlib import Path
from typing import Optional
from collections import Counter
```

| Library | Purpose in this pipeline |
|---|---|
| `subprocess` | Runs `pip install` commands programmatically to install missing packages |
| `sys` | Accesses `sys.executable` — the current Python interpreter path, needed to install packages into the correct environment |
| `os` | File path operations — `os.path.join()` , `os.path.isdir()` , `os.listdir()` |
| `glob` | Pattern matching for files — `glob.glob("*.dem")` finds all demo files |
| `warnings` | Suppresses noisy sklearn/numpy warnings that clutter output |

---

| Library | Purpose in this pipeline |
|---|---|
| `traceback` | Prints detailed error traces when demo parsing fails |
| `time` | `time.sleep(3)` — gives Google Drive 3 seconds to finish mounting |
| `Path` | Object-oriented file paths — `Path("folder") / "file.parquet"` |
| `Optional` | Type hints for function signatures |
| `Counter` | Counts occurrences — used in majority voting for ensemble clustering |

## Scientific Libraries

```
import numpy as np
import pandas as pd
from scipy.spatial import cKDTree
```

**NumPy ( `np` ):** The foundation of all numerical computing in Python. Used for:

- Array operations (vector math on player positions)
- `np.log1p()` — log transform for skewed features
- `np.clip()` — capping values to a range
- `np.linalg.norm()` — Euclidean distance calculations
- `np.nanpercentile()` — percentile calculations ignoring NaN values

**Pandas ( `pd` ):** Tabular data manipulation. Every player-round row is a row in a DataFrame with 14+ columns. Used for:

- Loading/saving Parquet files
- Filtering: `df[df["player_name"] == "ZywOo"]`
- Grouping: `df.groupby("cluster_id").mean()`
- `pd.cut()` — binning continuous values (assigning ticks to rounds)

**cKDTree (from scipy.spatial):** A k-dimensional tree for fast nearest-neighbor queries. Used to compute **mean isolation** — how far a player is from teammates. Without a KD-tree, computing nearest-neighbor distances for thousands of position samples would be $O(n^2)$. cKDTree does it in O(n log n).

## How a KD-Tree Works (In-Depth)

Imagine you have 100 teammate positions on a 2D map. For each of a player's 64 sampled positions, you need to find the closest teammate. Naively, that's 100 × 64 = 6,400 distance calculations.

A **KD-tree** is a binary tree that recursively splits the 2D space:

1. At the root, split all points by the median X coordinate
2. Left child: points with X < median. Right child: X ≥ median
3. At the next level, split by Y coordinate
4. Repeat, alternating X and Y

To query "what's the nearest teammate to position (500, 300)?", the tree prunes entire branches where no point could be closer than the current best. Instead of checking all 100 points, it typically checks ~7–10.

```
      Split on X=450
       /           \
  X<450            X≥450
  Split Y=200      Split Y=350
  /       \        /       \
...       ...    ...       ...
```

In our code:

```
tree = cKDTree(mate_xy)          # Build tree from teammate positions
dists, _ = tree.query(own_xy, k=1)  # Find nearest teammate for each play
return float(np.mean(dists))     # Average distance = mean isolation
```

## Machine Learning Libraries

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, confusion_matrix
from sklearn.manifold import TSNE
from sklearn.mixture import GaussianMixture
```

| Import | What it does | Where used |
|---|---|---|
| `StandardScaler` | Transforms features to mean=0, std=1 | Preprocessing (G1 Cell 5) |
| `KMeans` | Centroid-based clustering | Ensemble algorithm #1 |
| `PCA` | Linear dimensionality reduction | 14D → 2D for visualization |

| Import | What it does | Where used |
|---|---|---|
| `silhouette_score` | Measures cluster separation quality | Choosing optimal K |
| `confusion_matrix` | Predicted vs true label comparison | Validation (if ground truth available) |
| `TSNE` | Non-linear dimensionality reduction | Better 2D visualization than PCA |
| `GaussianMixture` | Probabilistic clustering (GMM) | Ensemble algorithm #2 |

## Visualization

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from tqdm import tqdm
```

- **matplotlib:** Base plotting library — scatter plots, bar charts, radar charts
- **mpatches:** Custom legend entries for cluster labels
- **seaborn:** Higher-level plotting built on matplotlib — heatmaps, color palettes
- **tqdm:** Progress bars — shows `Parsing demos: 45%|████      | 5/12`

## Optional Dependencies

```
try:
    import hdbscan
    HDBSCAN_AVAILABLE = True
except ImportError:
    HDBSCAN_AVAILABLE = False
```

**Why try/except?** HDBSCAN requires C extensions that might not compile on all systems. If it fails, the pipeline gracefully falls back to KMeans+GMM only (2-algorithm ensemble instead of 3).

```
try:
    from demoparser2 import DemoParser
    PARSER_AVAILABLE = True
except ImportError:
    PARSER_AVAILABLE = False
```

**demoparser2** is a Rust-based parser that reads CS2's proprietary `.dem` binary format. If unavailable, the pipeline cannot parse demos at all.

---

# 1.3 — Google Drive Mounting

```
try:
    from google.colab import drive
    try:
        drive.flush_and_unmount()
    except:
        pass
    drive.mount('/content/drive', force_remount=True)
    time.sleep(3)
    print("✅ Google Drive mounted.")
except ImportError:
    print("ℹ Not running on Colab — assuming Drive accessible.")
```

**What this does:**

1. Checks if running on Google Colab (by trying to import `google.colab`)
2. If yes: unmounts any stale Drive connection, then remounts fresh
3. `force_remount=True` ensures a clean connection (avoids "Mountpoint already contains files" errors)
4. `time.sleep(3)` — Google Drive mount is asynchronous. Without this pause, subsequent file access might fail because the filesystem hasn't fully synced
5. If not on Colab: assumes the filesystem is already accessible (e.g., running locally)

**Why `flush_and_unmount()` first?** If a previous notebook cell mounted Drive, the mount point `/content/drive` already has files. Google Colab raises `ValueError: Mountpoint must not already contain files` if you try to mount again. Unmounting first clears this.

---

# 1.4 — Configuration Constants

```
_BASE       = "/content/drive/MyDrive/cs2-btp"
DEM_FOLDER  = os.path.join(_BASE, "raw_dems")
OUTPUT_DIR  = Path(os.path.join(_BASE, "processed"))

RANDOM_SEED       = 42
```

```
MIN_TICK           = 0
POS_DOWNSAMPLE_N   = 64
CAP_DAMAGE         = 500
CAP_KILLS          = 5
CAP_UTILITY        = 5
CAP_TRADE_KILLS    = 5
CAP_DISTANCE       = 5000
CAP_ISOLATION      = 2000
TRADE_WINDOW_TICKS = 320
```

## Path Configuration

| Constant | Value | Purpose |
|---|---|---|
| `_BASE` | `/content/drive/MyDrive/cs2-btp` | Root project folder in Google Drive |
| `DEM_FOLDER` | `…/raw_dems` | Where .dem files live |
| `OUTPUT_DIR` | `…/processed` | Where outputs (Parquet files, PNGs) are saved |

## What Are Ticks?

CS2 runs its game simulation at **64 ticks per second** (in competitive mode). A "tick" is a single simulation frame. In a 2-minute round:

```
2 minutes × 60 seconds × 64 ticks = 7,680 ticks per round
```

Every player's position, health, weapon state, etc. is recorded at each tick. A full match demo might have **150,000+ ticks**.

## Why Cap Features?

| Cap | Value | Reasoning |
|---|---|---|
| `CAP_DAMAGE` | 500 | A player can deal ~400 damage max (4 kills × 100 HP + armor damage). 500 provides headroom for multi-kill rounds with penetration damage |
| `CAP_KILLS` | 5 | An ace (killing all 5 enemies). More than 5 kills is impossible in a single round |
| `CAP_UTILITY` | 5 | A player can carry at most 4 grenades + 1 C4. 5 is the theoretical max |

---

| Cap | Value | Reasoning |
|---|---|---|
| `CAP_TRADE_KILLS` | 5 | **FIX 1:** Without this cap, trade_kills could reach 17 due to counting artifacts |
| `CAP_DISTANCE` | 5000 | ~5000 game units is roughly crossing the entire map. Anything beyond is measurement noise |
| `CAP_ISOLATION` | 2000 | ~2000 units is being on the opposite side of the map from all teammates |

```
TRADE_WINDOW_TICKS = 320
```

320 ticks = 320/64 = **5 seconds**. In CS2, a "trade kill" is when you kill an enemy within ~5 seconds of them killing your teammate. This avenges the teammate's death. The 5-second window is the standard used by professional CS2 analytics.

```
POS_DOWNSAMPLE_N = 64
```

Instead of reading player positions at all 150,000 ticks (which would create a massive DataFrame), we sample every 64th tick. This gives **1 position sample per second** — enough to track movement patterns while keeping data manageable.

```
RANDOM_SEED = 42
```

Ensures reproducibility. KMeans, GMM, and t-SNE all use random initialization. Setting a seed means running the code twice gives identical results.

---

# 1.5 — DEM File Format & Parsing

## What Is a .dem File?

A `.dem` file is a **binary recording** of an entire CS2 match. It contains:

- Every player's position, rotation, and velocity at every tick
- Every game event (kills, damage, grenade throws, round ends)
- Team information, scores, buy phases
- Server configuration

A single map demo is typically **100-200 MB**.

## Team Name Normalization

```
TEAM_MAP = {"TERRORIST": "T", "COUNTER_TERRORIST": "CT", "T": "T", "CT":
```

Different demo versions encode team names differently. Some say `"TERRORIST"`, others say `"T"`. This mapping normalizes all variations to `"T"` or `"CT"`.

## `_safe_parse_event` — Error-Safe Event Parser

```
def _safe_parse_event(parser, event, cols):
    try:
        df = parser.parse_event(event, player=cols)
        if df is None or df.empty:
            return pd.DataFrame()
        for col in ["team_name", "attacker_team_name", "user_team_name"]:
            if col in df.columns:
                df[col] = df[col].map(lambda x: TEAM_MAP.get(str(x).upper
        return df
    except Exception:
        return pd.DataFrame()
```

**What it does:**

1. Asks `demoparser2` to extract a specific event type (e.g., `"player_death"`) with additional player properties (e.g., `["X", "Y", "headshot"]`)
2. If the event doesn't exist in this demo or parsing fails → returns an empty DataFrame instead of crashing
3. Normalizes team names using `TEAM_MAP`

**Why `player=cols`?** The `player` parameter tells demoparser2 to include additional player properties alongside the event data. For a `player_death` event, the base data is (tick, attacker_name, user_name). Adding `player=["X", "Y", "headshot"]` also includes the victim's position and whether it was a headshot.

## `_safe_parse_ticks` — Position Data Parser

```
def _safe_parse_ticks(parser, props, ticks):
    try:
        df = parser.parse_ticks(props, ticks=ticks)
        return df if df is not None else pd.DataFrame()
```

```
    except Exception:
        return pd.DataFrame()
```

Unlike events (which happen at specific moments), tick data is **continuous** — every player has a position at every tick. `parse_ticks` extracts properties at specified tick numbers.

## `parse_dem_file` — The Main Parser Function

```
def parse_dem_file(dem_path):
    parser = DemoParser(dem_path)

    # 1. Kill events
    kills = _safe_parse_event(parser, "player_death",
                              ["X", "Y", "Z", "team_name", "flash_duratic
                               "headshot", "assistedflash", "assister_nam

    # 2. Damage events
    damage = _safe_parse_event(parser, "player_hurt", ["X", "Y", "team_na

    # 3. Position data (sampled every 64 ticks)
    sample_ticks = list(range(MIN_TICK + 1, max_tick, POS_DOWNSAMPLE_N))
    positions = _safe_parse_ticks(parser, ["X", "Y", "Z", "team_name"], s

    # 4. Grenade events
    grenades = pd.DataFrame()
    for ev in ["weapon_fire", "inferno_startburn", "smokegrenade_detonate
               "hegrenade_detonate", "flashbang_detonate"]:
        g = _safe_parse_event(parser, ev, ["team_name"])
        grenades = pd.concat([grenades, g], ignore_index=True)

    # 5. Round end events
    round_ends = _safe_parse_event(parser, "round_end", [])

    return dict(kills=kills, damage=damage, positions=positions,
                grenades=grenades, round_ends=round_ends)
```

This function extracts 5 types of data from a demo:

| Data Type | CS2 Event | What Each Row Represents |
|---|---|---|
| **kills** | `player_death` | One player killed another. Includes attacker, victim, weapon, headshot, position, flash assist info |

---

| Data Type | CS2 Event | What Each Row Represents |
|---|---|---|
| **damage** | `player_hurt` | One player damaged another. Includes damage amount (HP and armor) |
| **positions** | tick data | A player's XYZ coordinates at a sampled tick. ~2,300 rows per player per match |
| **grenades** | multiple events | A grenade was thrown/detonated. Includes smoke, flash, HE, molotov |
| **round_ends** | `round_end` | A round finished. The tick of each round end defines round boundaries |

## Kill Event Columns (FIX 3)

The kill event includes these critical columns for feature engineering:

```
tick              — game tick when the kill happened
attacker_name     — who got the kill
user_name         — who died (the victim)
headshot          — boolean, was it a headshot?
weapon            — "ak47", "awp", "usp_silencer", etc.
assister_name     — who assisted the kill (if anyone)
assistedflash     — boolean, was the assist via a flashbang?
flash_duration    — how long the victim was flashed (seconds)
```

**FIX 3 Context:** The old v2 code tried to count flash assists by checking `attacker_name + flash_duration > 0`. But `flash_duration` on the kill event is how long the *victim* was blinded, not whether the attacker used a flash. The correct approach (v3) checks `assister_name == player_name AND assistedflash == True`.

## Position Data Structure

After parsing, `positions` is a DataFrame like:

```
   tick    name       X        Y       Z     team_name
0    65    ZywOo   -1200.5    500.3   128.0      CT
1    65    apEX    -1150.2    480.1   128.0      CT
2    65    mezii    -800.7    200.5   128.0      CT
...
```

Each row = one player at one tick. With 10 players and ~2,300 sampled ticks per match, this gives ~23,000 rows per demo.

---

## 1.6 — Data Flow Summary

```
.dem file (binary, ~150 MB)
    │
    ▼
DemoParser (demoparser2, Rust)
    │
    ├── kills DataFrame      (~300 rows per match)
    ├── damage DataFrame     (~1,500 rows per match)
    ├── positions DataFrame  (~23,000 rows per match)
    ├── grenades DataFrame   (~500 rows per match)
    └── round_ends DataFrame (~30 rows per match)
    │
    ▼
Feature Engineering (next explanation)
    │
    ▼
14-feature player-round DataFrame (~3,000 rows per 12 demos)
```

---

# Key Takeaways for Writing This From Scratch

1. **Always use try/except** when parsing demos — files can be corrupted or use unexpected formats
2. **Downsample positions** — reading every tick creates unmanageable data
3. **Normalize team names** — different demo versions use different strings
4. **Cap all features** — outliers from parsing artifacts can corrupt clustering
5. **Use** `RANDOM_SEED` everywhere — reproducibility is critical for research

# Explanation 2: Round Reconstruction & Feature Engineering Helpers

**Covers:** Lines 173–313 of `code_edited.py`

---

## 2.1 — Round Reconstruction: Turning Ticks into Rounds

A CS2 match has 24–30+ rounds, but the demo file doesn't explicitly label events with round numbers. Instead, it records `round_end` events with their tick. We must reverse-engineer round boundaries from these ticks.

### `reconstruct_rounds()`

```
def reconstruct_rounds(round_ends_df, max_fallback_tick=150_000):
    if round_ends_df.empty or "tick" not in round_ends_df.columns:
        return pd.DataFrame({"round_end_tick": [max_fallback_tick], "rour
    ticks = sorted(round_ends_df["tick"].dropna().unique().tolist())
    return pd.DataFrame({"round_end_tick": ticks, "round_num": range(1, l
```

**What it does:**

1. Takes the raw `round_end` events (which just have tick numbers)
2. Sorts them chronologically
3. Assigns round numbers 1, 2, 3, ...

**Example output:**

```
   round_end_tick  round_num
0           12800          1
1           21500          2
2           33200          3
...
28         210000         29
```

**Fallback:** If the demo has no round_end events (corrupt file), it creates a single "round" spanning the entire match.

### `assign_round_num()` — Mapping Ticks to Rounds

```
def assign_round_num(tick_series, round_map):
    bins   = [0] + round_map["round_end_tick"].tolist()
    labels = round_map["round_num"].tolist()
    assigned = pd.cut(tick_series, bins=bins, labels=labels, right=True)
    return pd.to_numeric(assigned, errors="coerce").fillna(1).astype(int)
```

**The problem:** Every kill, damage, and position event has a `tick` value. We need to know which round that tick belongs to.

**The solution:** `pd.cut()` creates bins from the round end ticks and assigns each event tick to its bin (round).

**Visual explanation:**

```
Round boundaries:     0 ——— 12800 ——— 21500 ——— 33200 ——— ...
                      |  R1  |   R2   |   R3   |   R4   |

Kill at tick 15000:   |      | ← here |        |        |  → Round 2
Kill at tick 8000:    |← here|        |        |        |  → Round 1
Position at 25000:    |      |        |← here  |        |  → Round 3
```

`pd.cut()` does this automatically:

- `bins = [0, 12800, 21500, 33200, ...]` — the boundaries
- `labels = [1, 2, 3, ...]` — what to call each bin
- `right=True` — a tick exactly AT 12800 belongs to round 1 (inclusive right boundary)

---

# 2.2 — Zone Classification (FIX 2)

## The CS2 Map Layout

Every CS2 map has three main areas:

- **T-spawn side** (left on most minimaps) — where Terrorists start
- **Mid** (center) — the middle corridor/area
- **CT-spawn side** (right on most minimaps) — where Counter-Terrorists start

We classify each player's average X position into one of these zones.

## `compute_zone_primary()` — Fixed in v3

```
def compute_zone_primary(mean_x, all_x):
    """
    Classify a player's mean X position into map zones using the global
    X distribution (all ticks, all players, whole match set) as the refer
    Returns 1 (CT/right side), 2 (mid), or 3 (T/left side).
    """
    if pd.isna(mean_x) or len(all_x) < 10:
        return 2
    p33 = np.nanpercentile(all_x, 33)
    p66 = np.nanpercentile(all_x, 66)
    if mean_x < p33:
        return 3
    elif mean_x > p66:
        return 1
    else:
        return 2
```

## Why Percentiles Instead of Fixed Thresholds?

Each CS2 map has completely different coordinate systems:

- **Dust2:** X ranges from roughly -2500 to +2500
- **Mirage:** X ranges from roughly -3000 to +1500
- **Nuke:** X ranges from roughly -1500 to +2000

If we used fixed thresholds like "X < -1000 = T side", it would be correct on Dust2 but wrong on Mirage.

**The percentile approach** is map-adaptive:

1. Collect ALL X positions from ALL players in the entire match ( `all_x` )
2. Calculate the 33rd and 66th percentiles — these naturally divide the map into thirds
3. Players in the bottom third → T-side (zone 3), middle third → mid (zone 2), top third → CT-side (zone 1)

## The Bug in v2 (FIX 2)

The old code called `compute_zone_adaptive([mean_x])` — passing a **single-element list**. The function had a guard: `if len(positions) < 10: return 2` . Since `len([mean_x]) = 1 <`

`10`, it ALWAYS returned zone 2 for every player. This bug made the `zone_primary` feature completely useless.

The fix: pass the **global** X distribution (`all_x` — tens of thousands of values) as the reference, and only classify `mean_x` against it.

---

# 2.3 — Mean Isolation (cKDTree-Based)

## What Is "Isolation" in CS2?

Isolation measures how far a player positions themselves from their teammates. High isolation = playing alone (lurking). Low isolation = staying with the pack (supporting).

### `mean_isolation()`

```
def mean_isolation(positions_df, player_name, round_num):
    rnd_pos = positions_df[positions_df["round_num"] == round_num]

    # Get this player's positions
    own = rnd_pos[rnd_pos[name_col] == player_name]
    own_xy = own[["X", "Y"]].dropna().values

    # Get this player's team
    own_team = own["team_name"].iloc[0]

    # Get teammate positions (same team, different player)
    mates = rnd_pos[(rnd_pos["team_name"] == own_team) &
                    (rnd_pos[name_col] != player_name)]
    mate_xy = mates[["X", "Y"]].dropna().values

    # Build a KD-tree from teammate positions
    tree = cKDTree(mate_xy)

    # For each of the player's positions, find the nearest teammate
    dists, _ = tree.query(own_xy, k=1)

    # Return the average nearest-teammate distance
    return float(np.mean(dists))
```

## Step-by-Step Example

Imagine Round 5 on Dust2. Player "s1mple" is on the T-side with 4 teammates.

**s1mple's positions (sampled every second):**

```
Tick 1: (1200, 800)   — near T-spawn
Tick 2: (1400, 600)   — moving toward Long
Tick 3: (1800, 300)   — Long Doors
Tick 4: (2200, 100)   — Long A site approach
```

**Teammate positions (all 4 teammates, all ticks combined):**

```
(600, 400), (650, 380), (700, 450), ...   — teammates at B/mid
```

The KD-tree is built from ALL teammate positions (maybe 200+ data points). For each of s1mple's 4 positions, it finds the nearest teammate:

```
Tick 1: nearest teammate at 580 units away
Tick 2: nearest teammate at 720 units away
Tick 3: nearest teammate at 1100 units away  ← getting more isolated
Tick 4: nearest teammate at 1500 units away  ← very isolated
```

**Mean isolation = (580 + 720 + 1100 + 1500) / 4 = 975 units**

A support player staying with the team might have mean isolation of ~200 units. A lurker might have ~1200+ units.

### Why Only Same-Team Teammates?

We only measure distance from **teammates**, not enemies. A player could be far from teammates but close to enemies (flanking). We want to capture "how far from your own team are you playing?" — that's what distinguishes lurkers from support players.

---

## 2.4 — Distance Traveled

```python
def distance_traveled(pos_rows):
    xy = pos_rows[["X", "Y"]].dropna().values
    if len(xy) < 2:
        return 0.0
    return float(np.sum(np.linalg.norm(np.diff(xy, axis=0), axis=1)))
```

### What This Computes

Total path length — the sum of Euclidean distances between consecutive position samples.

### Mathematical Breakdown

Given positions: $P_1$=(100,200), $P_2$=(150,250), $P_3$=(300,250), $P_4$=(350,300)

```
np.diff(xy, axis=0)   # Differences between consecutive positions
# = [(50,50), (150,0), (50,50)]

np.linalg.norm(..., axis=1)   # Euclidean distance of each step
# = [√(50²+50²), √(150²+0²), √(50²+50²)]
# = [70.7, 150.0, 70.7]

np.sum(...)   # Total distance
# = 291.4 game units
```

### Why 2D Not 3D?

We ignore the Z axis (height) because:

1. Most CS2 maps are played on a single elevation level
2. Vertical movement (jumping, climbing) is noise — it doesn't indicate tactical positioning
3. Ladders/stairs would inflate distance for players who simply change floors

### What Distance Tells Us About Roles

| Role | Typical Distance | Why |
|---|---|---|
| **Lurker** | 2000–4000+ | Covers large map areas, flanking through different routes |
| **Entry** | 1500–3000 | Rushes toward sites aggressively |
| **Support** | 800–1500 | Follows the entry, stays closer to starting position |
| **AWPer** | 500–1200 | Holds angles from static positions, moves less |
| **IGL** | 800–1500 | Moderate movement, directing rather than fragging |

---

## 2.5 — Trade Kills (FIX 1)

### What Is a Trade Kill?

In CS2, a "trade kill" happens when:

1. Your teammate dies
2. Within **5 seconds**, you kill the enemy who killed them (or any enemy nearby)

Trade kills are a key metric for **support players** — they're the ones who "trade" for fallen teammates, ensuring the team doesn't lose engagements for free.

## `compute_trade_kills()`

```
def compute_trade_kills(kills_df, player_name, round_num, side):
    rnd_kills = kills_df[kills_df["round_num"] == round_num]

    # Find teammate deaths (same team, not this player)
    mate_deaths = rnd_kills[
        (rnd_kills["user_team_name"] == side) &
        (rnd_kills["user_name"] != player_name)
    ]

    # Find this player's kills
    player_kills = rnd_kills[rnd_kills["attacker_name"] == player_name]

    # Count trades: player kill within 320 ticks (5 sec) of a mate death
    trades = 0
    for _, pk in player_kills.iterrows():
        for _, md in mate_deaths.iterrows():
            if 0 < (pk["tick"] - md["tick"]) <= TRADE_WINDOW_TICKS:
                trades += 1
                break   # Each kill can only be one trade

    return min(trades, CAP_TRADE_KILLS)   # FIX 1: cap at 5
```

## The Bug in v2 (FIX 1)

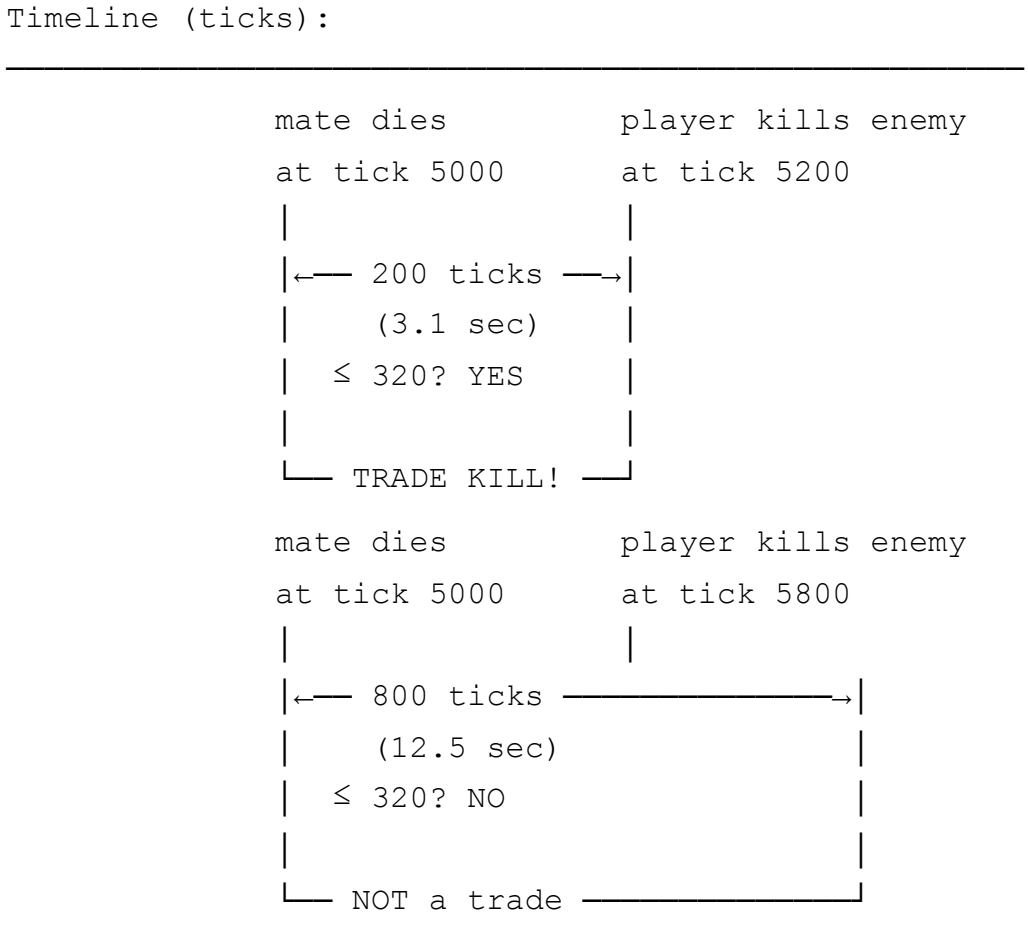
v2 had no `CAP_TRADE_KILLS` constant and no `min()` cap. In edge cases, the nested loop could count the same kill as multiple trades against multiple teammate deaths, producing values like 17. Since all other features are capped at ≤5, a trade_kills value of 17 would dominate clustering and distort results.

## The `break` Statement Is Critical

```
    for _, pk in player_kills.iterrows():       # For each of the player's kil
        for _, md in mate_deaths.iterrows():    # Check all teammate deaths
            if 0 < (pk["tick"] - md["tick"]) <= 320:
                trades += 1
                break    # ← This kill is a trade. Stop checking more deaths.
```

Without `break`, a single kill that happens after 2 teammate deaths (both within 5 seconds) would be counted as 2 trades. With `break`, it's correctly counted as 1 trade.

### Time Window Visualization




---

## 2.6 — Flash Assists (FIX 3)

### What Is a Flash Assist?

A flash assist happens when:

1. Player A throws a flashbang grenade
2. Enemy B gets blinded by it
3. Player C (teammate of A) kills Enemy B while B is still blinded

4. Player A gets credit for a "flash assist"

Flash assists are a key metric for **support players** — they flash for their entry fraggers.

## `compute_flash_assists()` — Fixed in v3

```
def compute_flash_assists(kills_df, player_name, round_num):
    rnd = kills_df[kills_df["round_num"] == round_num]

    if "assister_name" in rnd.columns and "assistedflash" in rnd.columns:
        fa = rnd[
            (rnd["assister_name"] == player_name) &
            (rnd["assistedflash"].fillna(False).astype(bool))
        ]
        return min(len(fa), CAP_UTILITY)

    # Fallback for older demo formats
    if "flash_duration" in rnd.columns and "attacker_name" in rnd.columns:
        fa = rnd[
            (rnd["attacker_name"] == player_name) &
            (rnd["flash_duration"].fillna(0) > 0)
        ]
        return min(len(fa), CAP_UTILITY)
    return 0
```

## The Bug in v2 (FIX 3)

v2 code checked: `attacker_name == player_name AND flash_duration > 0`

This is wrong because:

- `attacker_name` is who **got the kill**, not who threw the flash
- `flash_duration` on the kill event is how long the **victim** was blinded

So v2 was asking: "did this player kill someone while the victim was flashed?" — which is an "I killed a flashed enemy" metric, not "I flashed FOR someone else's kill."

v3 correctly uses:

- `assister_name` — the player who threw the flash
- `assistedflash` — boolean confirming it was a flash assist (not a damage assist)

---

# 2.7 — Opening Duel

## What Is an Opening Duel?

The **first kill** of each round is the "opening duel." The player who gets the opening kill has won the opening duel (+1). The player who dies first has lost it (-1). All other 8 players get 0.

```
def compute_opening_duel(kills_df, player_name, round_num):
    rnd = kills_df[kills_df["round_num"] == round_num].sort_values("tick"
    if rnd.empty:
        return 0
    first = rnd.iloc[0]   # The first kill of the round
    if first.get("attacker_name") == player_name:
        return 1           # Won the opening duel
    if first.get("user_name") == player_name:
        return -1          # Lost the opening duel (died first)
    return 0               # Wasn't involved
```

## Why Opening Duels Matter for Roles

| Role | Expected Opening Duel Pattern |
|---|---|
| **Entry** | Frequently +1 or -1 (always taking the first fight) |
| **AWPer** | Often +1 (gets first pick from holding an angle) |
| **Lurker** | Usually 0 (not involved in early fights) |
| **Support** | Usually 0 (following behind the entry) |
| **IGL** | Usually 0 (directing, not fragging first) |

Entry fraggers have the most volatile opening duel stats — they either win or lose the first fight. Support/Lurker/IGL players rarely participate in the opening duel.

---

# 2.8 — Feature Engineering Design Decisions

## Why 14 Features? Why These Specific Ones?

The 14 features were chosen to capture **distinct behavioral dimensions** that differentiate CS2 roles:

| Dimension | Features | What It Measures |
|---|---|---|
| **Aggression** | damage_dealt, kill_count, multi_kill_round | Raw fragging power |
| **Timing** | first_engagement_rel, death_tick_rel | When in the round the player acts |
| **Positioning** | mean_isolation, distance_traveled, zone_primary | Where the player plays |
| **Teamplay** | utility_used, trade_kills, flash_assists | How much they support teammates |
| **Precision** | headshot_pct | Weapon accuracy profile (rifler vs AWPer) |
| **Survivability** | survived | Whether the player lives through the round |
| **Initiative** | opening_duel_won | Who takes/wins the first fight |

Each feature captures a different aspect of gameplay. Together, they create a 14-dimensional behavioral fingerprint for each player-round.

---

# Key Takeaways for Writing This From Scratch

1. **Round reconstruction** is the foundation — without correct round boundaries, all per-round features are wrong
2. **Use `pd.cut()` for binning** — it's the Pandas-native way to assign continuous values to discrete bins
3. **Zone classification must be map-adaptive** — never use hardcoded X thresholds
4. **KD-trees** make isolation computation feasible — $O(n \log n)$ vs $O(n^2)$
5. **Cap everything** — one uncapped outlier can dominate StandardScaler and corrupt clustering
6. **Read the demoparser2 column names carefully** — `assister_name ≠ attacker_name`, `assistedflash ≠ flash_duration`

# Explanation 3: Feature Extraction Loop & Preprocessing

**Covers:** Lines 315–568 of `code_edited.py`

---

## 3.1 — The Main Extraction Function: `extract_features_from_dem()`

This is the most complex function in the pipeline. It takes a single `.dem` file and produces a DataFrame where **each row = one player in one round** with 14 behavioral features.

### High-Level Flow

```
.dem file
    |
    ▼
parse_dem_file()  →  kills, damage, positions, grenades, round_ends
    |
    ▼
reconstruct_rounds()  →  round_map (round boundaries)
    |
    ▼
assign_round_num() applied to all DataFrames  →  each event tagged with round_num
    |
    ▼
Collect unique (player, round, side) combos
    |
    ▼
For EACH combo: compute 14 features
    |
    ▼
Output: DataFrame with ~300 rows (10 players × 30 rounds)
```

## Step 1: Parse Events and Tag Rounds

```
def extract_features_from_dem(dem_path, match_id):
    events = parse_dem_file(dem_path)
    kills_df, damage_df = events["kills"], events["damage"]
    pos_df, grenades_df, re_df = events["positions"], events["grenades"],
    round_map = reconstruct_rounds(re_df)

    for df in [kills_df, damage_df, pos_df, grenades_df]:
        if not df.empty and "tick" in df.columns:
            df["round_num"] = assign_round_num(df["tick"], round_map)
```

This assigns a `round_num` column to every event. Now every kill, damage event, position sample, and grenade throw knows which round it belongs to.

## Step 2: Build Global X Distribution (FIX 2)

```
all_x = pos_df["X"].dropna().values if not pos_df.empty and "X" in pos_df
```

This collects ALL X coordinates from ALL players across the ENTIRE match. With ~23,000 position samples, `all_x` has ~23,000 values. This large distribution is used by `compute_zone_primary()` to calculate meaningful percentiles.

## Step 3: Discover Player-Round Combos

```
combos = set()
for df, hint in [(kills_df, "attacker_name"), (damage_df, "attacker_name"
    for _, row in df.iterrows():
        pname = row.get(nc)
        rnd   = row.get("round_num", 1)
        side  = row.get(tc, "T")
        combos.add((str(pname), int(rnd), str(side)))

# Also add VICTIMS (so players with 0 kills are included)
if not kills_df.empty and "user_name" in kills_df.columns:
    for _, row in kills_df.iterrows():
        pname = row.get("user_name")
        combos.add((str(pname), int(rnd), str(side)))
```

**Why use a set?** A player might appear multiple times in the kills DataFrame (multiple kills in one round). Using a `set` ensures each (player, round, side) combination is processed exactly once.

**Why include victims?** A player who dies without getting any kills still has valid behavioral data (position, isolation, death timing, utility). Without including victims, these passive rounds would be lost — biasing the dataset toward aggressive plays.

**What's a "combo"?** A tuple like `("ZywOo", 5, "CT")` meaning "ZywOo in round 5, playing on CT side."

## Step 4: Compute 14 Features Per Combo

For each (player, round, side) combo, the code computes:

**Feature 1:** `kill_count`

```
kdf = kills_df[(kills_df["attacker_name"] == player_name) &
               (kills_df["round_num"] == round_num)]
kill_count = min(len(kdf), CAP_KILLS)   # Cap at 5 (ace)
```

Filter the kills DataFrame for this player's kills in this round. Count them.

**Feature 2:** `headshot_pct`

```
headshot_pct = float(kdf["headshot"].sum() / kill_count)
```

What fraction of kills were headshots. The `headshot` column is boolean (True/False), so `.sum()` counts True values.

- Riflers (AK-47, M4): ~50-65% headshot rate (one-tap headshots)
- AWPers: ~10-25% (AWP kills are usually body shots — one shot kill anyway)

**Feature 3:** `damage_dealt`

```
ddf = damage_df[(damage_df["attacker_name"] == player_name) &
                (damage_df["round_num"] == round_num)]
damage_dealt = float(min(ddf["dmg_health"].sum(), CAP_DAMAGE))
```

Sum all HP damage this player dealt in this round. Each hit is a separate row in the damage DataFrame.

**Fallback:** If no damage data exists, estimate as `kill_count × 80` (average damage per kill).

**Feature 4:** `utility_used`

```
gdf = grenades_df[(grenades_df[nc] == player_name) &
                  (grenades_df["round_num"] == round_num)]
utility_used = min(len(gdf), CAP_UTILITY)
```

Count grenade events by this player in this round.

**Features 5-6:** `distance_traveled` **&** `mean_isolation`

```
player_pos = pos_df[(pos_df[nc] == player_name) & (pos_df["round_num"] ==
dist = min(distance_traveled(player_pos), CAP_DISTANCE)
iso  = min(mean_isolation(pos_df, player_name, round_num), CAP_ISOLATION)
```

Both use the position data, as explained in Explanation 2.

**Feature 7:** `zone_primary`

```
mean_x = player_pos["X"].mean()
zone_prim = compute_zone_primary(mean_x, all_x)   # FIX 2: global referer
```

The player's average X position, classified into zone 1 (CT), 2 (mid), or 3 (T).

**Features 8-9:** `first_engagement_rel` **&** `death_tick_rel`

```
# Calculate round boundaries
rnd_end_tick = round_map[round_map["round_num"] == round_num]["round_end_
prev = round_map[round_map["round_num"] == round_num - 1]["round_end_tick
rnd_start_tick = prev.iloc[0] if not prev.empty else 0
rnd_len = max(rnd_end_tick - rnd_start_tick, 1)

# First kill timing (relative to round length)
first_kill_tick = kills_df[(kills_df["attacker_name"] == player_name) &
                           (kills_df["round_num"] == round_num)]["tick"].
fe_rel = float(np.clip((first_kill_tick - rnd_start_tick) / rnd_len, 0, 1

# Death timing (relative to round length)
death_tick = kills_df[(kills_df["user_name"] == player_name) &
                      (kills_df["round_num"] == round_num)]["tick"].min()
dt_rel = float(np.clip((death_tick - rnd_start_tick) / rnd_len, 0, 1))
```

**Both features are normalised to [0, 1]:**

- `0.0` = happened at the very start of the round
- `0.5` = happened at the midpoint
- `1.0` = happened at the very end (or never happened — see below)

**Default value = 1.0** if the event didn't occur:

- `fe_rel = 1.0` when the player got zero kills → "never engaged" (late)
- `dt_rel = 1.0` when the player survived → "never died" (survived the entire round)

**Why this makes sense for roles:**

| Role | first_engagement_rel | death_tick_rel |
|---|---|---|
| Entry | ~0.1–0.3 (early kills) | ~0.1–0.3 (dies early) |
| Lurker | ~0.5–0.8 (late kills) | ~0.7–1.0 (survives) |
| AWPer | ~0.2–0.5 (picks from angles) | ~0.5–1.0 (varies) |
| Support | ~0.3–0.6 (follows entry) | ~0.3–0.7 (moderate) |

**Features 10-14: The New v3 Features**

```
trade_kills      = compute_trade_kills(kills_df, player_name, round_num,
flash_assists    = compute_flash_assists(kills_df, player_name, round_num
multi_kill_round = 1 if kill_count >= 3 else 0
survived         = 1 if np.isnan(death_tick) else 0
opening_duel_won = compute_opening_duel(kills_df, player_name, round_num)
```

| Feature | Type | Value Range | What It Captures |
|---|---|---|---|
| `trade_kills` | int | 0–5 | Teamplay: avenging teammates |
| `flash_assists` | int | 0–5 | Teamplay: flashing for kills |
| `multi_kill_round` | binary | 0 or 1 | Star power: 3+ kills in a round |
| `survived` | binary | 0 or 1 | Survivability |
| `opening_duel_won` | ternary | -1, 0, or 1 | Initiative: won/lost first fight |

## Step 5: Assemble the Row

```
    rows.append({
        "match_id": match_id, "player_name": player_name,
        "round_num": round_num, "side": side,
        "kill_count": kill_count, "damage_dealt": damage_dealt,
        "utility_used": utility_used, "distance_traveled": dist,
        "mean_isolation": iso, "first_engagement_rel": fe_rel,
        "death_tick_rel": dt_rel, "zone_primary": zone_prim,
        "headshot_pct": headshot_pct, "trade_kills": trade_kills,
        "flash_assists": flash_assists, "multi_kill_round": multi_kill_round,
        "survived": survived, "opening_duel_won": opening_duel_won,
    })
```

Each row is a dictionary that becomes one row in the final DataFrame. The four metadata columns ( `match_id` , `player_name` , `round_num` , `side` ) are NOT used in clustering — they're only for identification and later aggregation.

---

## 3.2 — Parsing All Demos

```
dem_files = sorted(glob.glob(os.path.join(DEM_FOLDER, "*.dem")))

all_frames = []
for dem_path in tqdm(dem_files, desc="Parsing demos"):
    match_id = Path(dem_path).stem   # "vitality_vs_spirit_dust2"
    df = extract_features_from_dem(dem_path, match_id)
    if not df.empty:
        all_frames.append(df)

features_df = pd.concat(all_frames, ignore_index=True)
```

**`Path(dem_path).stem`** extracts the filename without extension:

- `/content/drive/MyDrive/cs2-btp/raw_dems/vitality_vs_spirit_dust2.dem` → `"vitality_vs_spirit_dust2"`

**`pd.concat(all_frames, ignore_index=True)`** stacks all per-demo DataFrames vertically:

```
Demo 1: 300 rows (10 players × 30 rounds)
Demo 2: 280 rows
Demo 3: 310 rows
...
```

```
Total: ~3,600 rows × 18 columns
```

`ignore_index=True` resets the index to 0, 1, 2, ... instead of keeping each demo's internal index.

---

# 3.3 — Preprocessing & Scaling

## Why Preprocess?

Raw features have wildly different scales:

- `damage_dealt` : 0 to 500 (large numbers)
- `headshot_pct` : 0 to 1.0 (tiny numbers)
- `distance_traveled` : 0 to 5000 (huge numbers)
- `survived` : 0 or 1 (binary)

If we feed these directly to KMeans, `distance_traveled` would dominate because its values are ~5000× larger than `headshot_pct` . KMeans uses Euclidean distance, and a difference of 100 in distance_traveled would "matter" 500× more than a difference of 0.2 in headshot_pct. This is wrong — both features should have equal influence.

## Three Feature Categories

```
LOG_FEATURES    = ["damage_dealt", "distance_traveled", "mean_isolation"]
COUNT_FEATURES  = ["kill_count", "utility_used", "trade_kills", "flash_as
BINARY_FEATURES = ["multi_kill_round", "survived"]
PASSTHROUGH     = ["first_engagement_rel", "death_tick_rel", "zone_primar
                   "headshot_pct", "opening_duel_won"]
```

| Category | Transform | Why |
|---|---|---|
| **LOG features** | `log1p()` → `StandardScaler` | These features are right-skewed (many low values, few high values). Log transform makes them more normal |
| **COUNT features** | `StandardScaler` only | Small integer ranges (0–5), already roughly symmetric |
| **BINARY features** | `StandardScaler` only | 0/1 values, scaling centers them at 0 |
| **PASSTHROUGH** | No transform (already [0,1] or small range) | Already on a reasonable scale |

## Log Transform: `np.log1p()`

### What `log1p` Does

`log1p(x) = ln(x + 1)` — the natural logarithm of (x + 1).

The "+1" is critical: `ln(0)` is undefined (−∞), but `ln(0 + 1) = ln(1) = 0`. So `log1p(0) = 0`, which is safe.

### Why Log Transform?

`damage_dealt` has a heavily right-skewed distribution:

```
Without log:
0-50     ████████████████████████████  (70% of values)
50-100   ██████████                    (20%)
100-200  ████                          (7%)
200-500  ▌                             (3%)  ← outliers dominate

With log1p:
0-2      ████████████████              (35%)
2-3.5    ████████████████              (35%)
3.5-5    ███████████                   (22%)
5-6.2    ████                          (8%)   ← much more balanced
```

Log compresses large values and spreads out small values, making the distribution more **symmetric/normal**. This is important because StandardScaler assumes roughly normal distributions.

### Concrete Example

```
damage values: [0, 15, 30, 45, 80, 120, 200, 450]

After log1p:   [0, 2.77, 3.43, 3.83, 4.39, 4.80, 5.30, 6.11]
                ← Much more evenly spaced
```

## StandardScaler: Z-Score Normalization

```
scaler = StandardScaler()
scaled_vals = scaler.fit_transform(df[to_scale].fillna(0))
```

**What StandardScaler Does**

For each feature column, it transforms every value to a **z-score**:

$z = (x - \mu) / \sigma$

Where:

- $x$ = the original value
- $\mu$ = the mean of that feature across all rows
- $\sigma$ = the standard deviation of that feature

**After Scaling**

- Mean of each feature = 0
- Standard deviation = 1
- A value of +2.0 means "2 standard deviations above average"
- A value of -1.5 means "1.5 standard deviations below average"

**Why This Matters for Clustering**

Before scaling:

```
Player A: damage=300, distance=4000
Player B: damage=280, distance=100

Euclidean distance = √((300-280)² + (4000-100)²) = √(400 + 15,210,000) ≈ 3900
                                                    ↑ distance dominates!
```

After scaling (assuming μ_dmg=150, σ_dmg=80, μ_dist=2000, σ_dist=1500):

```
Player A: damage_scaled=(300-150)/80=1.88, distance_scaled=(4000-2000)/1500=1.33
Player B: damage_scaled=(280-150)/80=1.63, distance_scaled=(100-2000)/1500=-1.27

Euclidean distance = √((1.88-1.63)² + (1.33-(-1.27))²) = √(0.0625 + 6.76) ≈ 2.61
                                                          ↑ both features contribute
```

## Passthrough Features

```
for col in PASSTHROUGH:
    df[f"{col}_scaled"] = df[col].astype(float)
```

These features are already on [0,1] or [-1,1] scales. They get a `_scaled` suffix (for column naming consistency) but **no actual transformation**. This is a design choice — passthrough features are already comparable in scale to z-scored features (which are typically in [-3, +3]).

---

# 3.4 — Validation Report

```
raw_check = {
    "damage_dealt":          (0, CAP_DAMAGE),
    "kill_count":            (0, CAP_KILLS),
    ...
}
for col, (lo, hi) in raw_check.items():
    mn, mx = clean_df[col].min(), clean_df[col].max()
    ok = "✅" if (mn >= lo - 1e-6 and mx <= hi + 1e-6) else "❌"
```

This sanity check verifies that every feature is within its expected range after preprocessing. If any feature fails:

- Capping might have been missed
- A bug in feature computation produced out-of-range values
- Demo parsing returned unexpected data

The `1e-6` tolerance accounts for floating-point precision errors (e.g., 0.99999999 vs 1.0).

## Additional Validation Checks

```
print(f"zone_primary distribution:\n{clean_df['zone_primary'].value_count
print(f"flash_assists > 0: {(clean_df['flash_assists'] > 0).sum()} rows")
```

- **zone_primary distribution** should show roughly equal counts for zones 1, 2, 3 (if FIX 2 worked). If all values are 2, the fix didn't work.
- **flash_assists > 0** should be non-zero. If it's 0, FIX 3 didn't work.

---

# 3.5 — Output: The 14-Feature DataFrame

The final `clean_df` looks like this:

```
match_id           player_name  round_num  side  kill_count  damage_dealt
...  damage_dealt_log_scaled  kill_count_scaled  ...
vitality_spirit_d2      ZywOo          1     CT           2         180.0
...                    1.42              0.85  ...
vitality_spirit_d2      ZywOo          2     CT           0           0.0
...                   -1.23             -1.10  ...
vitality_spirit_d2        apEX         1     CT           3         250.0
...                    1.89              1.62  ...
...
```

There are 3 types of columns:

1. **Metadata** (4): `match_id`, `player_name`, `round_num`, `side` — for identification
2. **Raw features** (14): `kill_count`, `damage_dealt`, ... — original values
3. **Scaled features** (14): `kill_count_scaled`, `damage_dealt_log_scaled`, ... — for clustering

Only the 14 scaled columns are fed to clustering algorithms. The rest are kept for profiling and interpretability.

---

# Key Takeaways for Writing This From Scratch

1. **Build a combo set first** — don't iterate through events directly, build (player, round, side) tuples first to avoid duplicates
2. **Include victims** — players with 0 kills still have behavioral data
3. **Use log1p, not log** — `log(0)` crashes, `log1p(0) = 0` is safe
4. **Separate feature categories** — log, count, binary, passthrough each need different treatment
5. **Always validate** — check ranges, check distributions, check for all-zero columns
6. **Save intermediate results** — `g1_features.parquet` (raw) and `g1_clean.parquet` (scaled) allow restarting from either checkpoint

# Explanation 4: Clustering — K Selection, Ensemble, & Visualization

**Covers:** Lines 570–754 of `code_edited.py`

---

## 4.1 — What Is Clustering?

Clustering is **unsupervised learning** — finding natural groups in data **without** pre-defined labels. Unlike supervised learning (where you train on labeled examples), clustering discovers structure from the data itself.

In our pipeline:

- **Input:** 14-dimensional feature vectors (one per player-round)
- **Output:** A cluster ID (0, 1, 2, ..., K-1) for each player-round
- **Goal:** Each cluster should correspond to a CS2 role (Entry, Lurker, etc.)

We don't tell the algorithm what "Entry" looks like — it discovers that some player-rounds have high damage + early engagement + low survival and groups them together.

---

## 4.2 — Loading Scaled Features

```
SCALED_FEATURE_COLS = (
    [f"{c}_log_scaled" for c in LOG_FEATURES] +
    [f"{c}_scaled" for c in COUNT_FEATURES] +
    [f"{c}_scaled" for c in BINARY_FEATURES] +
    [f"{c}_scaled" for c in PASSTHROUGH]
)

g1_clean = pd.read_parquet(OUTPUT_DIR / "g1_clean.parquet")
X_scaled = g1_clean[SCALED_FEATURE_COLS].fillna(0).values
```

`X_scaled` is a numpy array of shape `(N, 14)` where:

- N ≈ 3,600 (total player-rounds across all demos)

- 14 = number of scaled features

Each row is a 14-dimensional vector. Clustering algorithms operate on this matrix.

`.fillna(0)` — replace any NaN values with 0. NaN can occur when a feature couldn't be computed (e.g., no position data for a player in a round). Using 0 after StandardScaler means "this player was exactly average on this feature."

---

# 4.3 — Feature Correlation Matrix

```
corr = g1_clean[SCALED_FEATURE_COLS].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0)
```

## What This Shows

A correlation matrix measures how much each pair of features moves together:

- **+1.0** = perfect positive correlation (when A goes up, B goes up)
- **-1.0** = perfect negative correlation (when A goes up, B goes down)
- **0.0** = no linear relationship

## Why It Matters for Clustering

If two features are highly correlated (e.g., `distance_traveled` and `mean_isolation` might be ~0.6), they're partially redundant. This means:

1. The clustering algorithm "double-counts" that behavioral dimension
2. Lurkers (high distance + high isolation) get extra weight in distance calculations
3. This can inflate certain cluster sizes

Ideally, all features should be <0.5 correlation. If two features are >0.8, consider dropping one.

## The `cmap="RdBu_r"` Color Scheme

- Red = positive correlation
- White = no correlation
- Blue = negative correlation
- `center=0` ensures white is at exactly 0 correlation

---

# 4.4 — Choosing Optimal K (Number of Clusters)

## The Problem

We need to decide how many clusters to create. Too few (K=2) → all roles merge together. Too many (K=10) → artificial splits within the same role.

## Method 1: Silhouette Score

```
for k in range(3, 8):
    km = KMeans(n_clusters=k, n_init=10, random_state=RANDOM_SEED)
    lbl = km.fit_predict(X_scaled)
    sil_scores[k] = silhouette_score(X_scaled, lbl, sample_size=min(5000,
```

### How Silhouette Score Works

For each data point i:

1. **a(i)** = average distance from i to all other points in its own cluster (intra-cluster distance)
2. **b(i)** = average distance from i to all points in the nearest OTHER cluster (nearest-cluster distance)
3. **s(i) = (b(i) - a(i)) / max(a(i), b(i))**

The score ranges from -1 to +1:

- **+1** = perfectly clustered (far from other clusters, close to own cluster)
- **0** = on the boundary between clusters
- **-1** = probably in the wrong cluster

The **overall silhouette score** is the mean of s(i) across all data points.

### Visual Intuition

```
Good clustering (high silhouette):
    ●●●●          ■■■■
    ●●●●          ■■■■     Two tight, well-separated groups
    ●●●●          ■■■■
    a(i) small    b(i) large

Bad clustering (low silhouette):
    ●●●■■●●
    ●■●●■●■●          Overlapping, mixed groups
```

■●■●●■●■

```
a(i) ≈ b(i) → s(i) ≈ 0
```

**We pick the K with the HIGHEST silhouette score** → best cluster separation.

## Method 2: BIC Score (Bayesian Information Criterion)

```
gmm = GaussianMixture(n_components=k, covariance_type="full", n_init=3)
gmm.fit(X_scaled)
bic_scores[k] = gmm.bic(X_scaled)
```

**How BIC Works**

BIC balances model fit vs complexity:

```
BIC = -2 × log-likelihood + p × ln(N)
```

Where:

- **log-likelihood** = how well the model fits the data (higher is better)
- **p** = number of parameters in the model (more clusters = more parameters)
- **N** = number of data points

BIC penalises models with more parameters. A model with K=20 clusters might fit the data perfectly, but BIC would penalise it for having too many parameters (overfitting).

**We pick the K with the LOWEST BIC** → best fit-to-complexity ratio.

**Why BIC Uses GMM, Not KMeans**

KMeans doesn't have a probabilistic model, so it can't compute likelihood. GMM (Gaussian Mixture Model) IS a probabilistic model — it explicitly computes P(data | model), which is needed for BIC.

## Consensus Between Silhouette and BIC

```
OPTIMAL_K_SIL = max(sil_scores, key=sil_scores.get)    # Highest silhouett
OPTIMAL_K_BIC = min(bic_scores, key=bic_scores.get)     # Lowest BIC
OPTIMAL_K = (OPTIMAL_K_SIL if abs(OPTIMAL_K_SIL - OPTIMAL_K_BIC) <= 1
             else max(3, min(7, round((OPTIMAL_K_SIL + OPTIMAL_K_BIC) / 2
```

**Logic:**

1. If both methods agree (or differ by ≤1) → use silhouette's choice

2. If they disagree significantly → average them, clamped to [3, 7]

**Example:** Silhouette says K=3 (best separation), BIC says K=7 (best fit). They disagree by 4, so we average: (3+7)/2 = 5.

In practice, the pipeline usually settles on **K=5** which aligns with the 5–6 tactical roles in CS2.

---

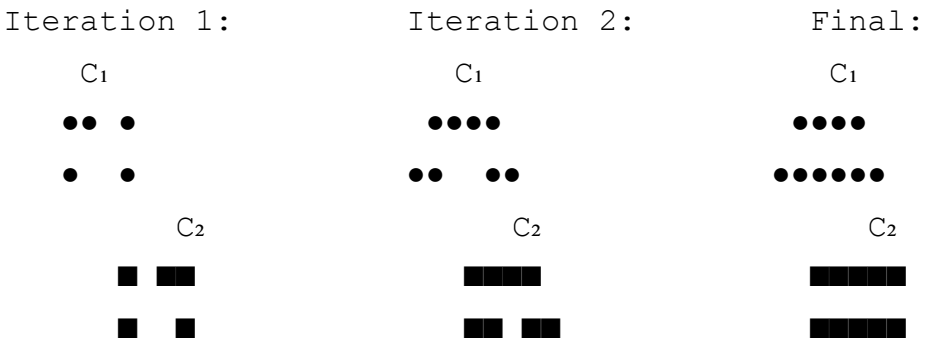
# 4.5 — The Three Clustering Algorithms

## Algorithm 1: KMeans

```
km_model = KMeans(n_clusters=OPTIMAL_K, n_init=10, random_state=RANDOM_SE
km_labels = km_model.fit_predict(X_scaled)
```

### How KMeans Works

1. **Initialize:** Randomly place K centroids in 14D space
2. **Assign:** Each data point → nearest centroid (by Euclidean distance)
3. **Update:** Move each centroid to the mean of its assigned points
4. **Repeat** steps 2-3 until centroids stop moving (convergence)




`n_init=10` — KMeans is sensitive to initial centroid placement. Running 10 times with different random starts and keeping the best result reduces this sensitivity.

### Strengths & Weaknesses

| ✅ Strengths | ❌ Weaknesses |
|---|---|
| Fast (O(NKI) per iteration) | Assumes spherical clusters |
| Always converges | Sensitive to outliers |
| Easy to interpret (centroids) | Must specify K in advance |
| Deterministic with fixed seed | Can't handle non-convex shapes |

## Algorithm 2: Gaussian Mixture Model (GMM)

```
gmm_model = GaussianMixture(n_components=OPTIMAL_K, covariance_type="full
                            n_init=5, random_state=RANDOM_SEED)
gmm_labels = gmm_model.fit_predict(X_scaled)
gmm_probs  = gmm_model.predict_proba(X_scaled)
```

**How GMM Works**

GMM assumes the data was generated by K Gaussian (normal) distributions, each with its own:

- **Mean (μ):** Center of the cluster (like KMeans centroid)
- **Covariance (Σ):** Shape and orientation of the cluster (KMeans assumes spherical)
- **Weight (π):** Relative size of the cluster

It uses the **Expectation-Maximization (EM)** algorithm:

1. **E-step:** For each data point, compute the probability it belongs to each Gaussian:

   $P(\text{point } i \text{ belongs to cluster } k) = \pi_k \times N(x_i \mid \mu_k, \Sigma_k) \;/\; \Sigma_j \pi_j \times N(x_i \mid \mu_j, \Sigma_j)$

2. **M-step:** Update each Gaussian's parameters using the weighted data points:

   $\mu_k$ = weighted mean of all points (weighted by P(belongs to k))
   $\Sigma_k$ = weighted covariance
   $\pi_k$ = fraction of total weight assigned to k

3. **Repeat** until convergence.

**Key Difference from KMeans: Soft Clustering**

KMeans: each point belongs to **exactly one** cluster (hard assignment).
GMM: each point has a **probability** of belonging to each cluster (soft assignment).

```
                KMeans          GMM
Point X:        Cluster 2       P(C0)=0.05, P(C1)=0.15, P(C2)=0.70,
P(C3)=0.10
```

The `gmm_probs` matrix stores these probabilities. `gmm_max_prob` = the highest probability for each point — a measure of **confidence**.

- `gmm_max_prob = 0.95` → very clearly belongs to one cluster
- `gmm_max_prob = 0.35` → borderline case, could belong to multiple clusters

`covariance_type="full"` means each cluster has its own full covariance matrix. This allows **elliptical** clusters (elongated, tilted), not just spherical ones like KMeans.

```
KMeans clusters:        GMM clusters:
    o
   ooo
    o
```

## Algorithm 3: HDBSCAN (Hierarchical DBSCAN)

```
hdb_model = hdbscan.HDBSCAN(
    min_cluster_size=max(20, len(X_scaled) // 100),
    min_samples=5,
    metric="euclidean"
)
hdb_labels_raw = hdb_model.fit_predict(X_scaled)
```

#### How HDBSCAN Works (Simplified)

1. **Compute mutual reachability distance:** For each pair of points, compute a modified distance that considers local density
2. **Build a minimum spanning tree:** Connect all points using shortest edges
3. **Build a cluster hierarchy:** Progressively remove edges (longest first), creating a tree of nested clusters
4. **Extract stable clusters:** Find clusters that persist across many distance thresholds (stable = "real" cluster)

#### Key Properties

- **No K required:** HDBSCAN automatically determines the number of clusters
- **Noise detection:** Points that don't belong to any dense region are labeled `-1` (noise)
- **Non-convex clusters:** Can find arbitrarily shaped clusters

`min_cluster_size=max(20, N//100)` **:** A cluster must have at least this many points. With 3,600 samples: `max(20, 36) = 36`. This prevents tiny noise clusters.

`min_samples=5` **:** How conservative the algorithm is. Higher = more points labeled as noise.

#### Noise Handling

```
unique_hdb = sorted(set(hdb_labels_raw) - {-1})   # Remove noise label
hdb_remap = {old: new for new, old in enumerate(unique_hdb)}
hdb_remap[-1] = -1
hdb_labels = np.array([hdb_remap[l] for l in hdb_labels_raw])
```

HDBSCAN might produce clusters labeled [0, 3, 7, -1] (non-consecutive, plus noise). This remaps them to [0, 1, 2, -1] for consistency.

---

# 4.6 — Ensemble Consensus: Combining Three Algorithms

## The Problem

KMeans gives labels [0,1,2,3,4], GMM gives labels [0,1,2,3,4], HDBSCAN gives labels [0,1,2,3,4,-1]. But **label numbers are arbitrary** — KMeans's "cluster 0" might be the same group as GMM's "cluster 3".

## Label Alignment: Hungarian Algorithm

```
def align_labels(ref, target, k):
    cost = np.zeros((k, k))
    for i in range(k):
        for j in range(k):
            cost[i, j] = -np.sum((ref == i) & (target == j))
    row_ind, col_ind = linear_sum_assignment(cost)
    mapping = {col_ind[i]: row_ind[i] for i in range(len(row_ind))}
    return np.array([mapping.get(l, l % k) for l in target])
```

### How the Hungarian Algorithm Works Here

We create a cost matrix where `cost[i][j]` = how many data points have `ref_label = i` AND `target_label = j`.

**Example:** With K=3 and 1000 data points:

```
                GMM label 0    GMM label 1    GMM label 2
KMeans label 0:      30             250             20
KMeans label 1:     280              10             15
KMeans label 2:      10              40            345
```

The Hungarian algorithm finds the optimal one-to-one mapping that maximises overlap:

- GMM 0 → KMeans 1 (overlap: 280)
- GMM 1 → KMeans 0 (overlap: 250)
- GMM 2 → KMeans 2 (overlap: 345)

So GMM labels are remapped: `{0→1, 1→0, 2→2}`. Now both algorithms use the same numbering.

**Why `-np.sum()` (Negative)?**

`linear_sum_assignment` MINIMISES cost. We want to MAXIMISE overlap, so we negate the overlap values. Minimising `-overlap` = maximising `overlap`.

## HDBSCAN Noise Handling

```
hdb_valid = hdb.copy()
hdb_valid[hdb_valid == -1] = km[hdb_valid == -1]   # Assign noise points
```

HDBSCAN marks some points as noise (-1). For ensemble voting, we can't have -1 votes. So noise points get assigned their KMeans label as a fallback.

## Majority Voting

```
consensus = np.zeros(len(km), dtype=int)
for i in range(len(km)):
    votes = [km[i], gmm_aligned[i], hdb_aligned[i]]
    consensus[i] = Counter(votes).most_common(1)[0][0]
```

For each data point, three algorithms vote on its cluster:

- If all 3 agree → consensus = that label (high confidence)
- If 2 agree, 1 disagrees → consensus = majority label
- If all 3 disagree → consensus = KMeans (since it's the reference)

```
Point #42:
  KMeans:  cluster 2
  GMM:     cluster 2  (aligned)
  HDBSCAN: cluster 4  (aligned)

  Votes: [2, 2, 4] → Counter: {2: 2, 4: 1} → most_common → 2
  Consensus: cluster 2 ✅
```

### Agreement Statistics

```
agree_all = np.mean(
    (km_labels == cluster_labels) &
    (gmm_aligned == cluster_labels) &
    (hdb_aligned == cluster_labels)
)
```

This reports what fraction of data points have all 3 algorithms agreeing. Typical values:

- **60-80%** agreement: Good — algorithms see similar structure
- **30-50%** agreement: Moderate — significant uncertainty
- **<20%** agreement: Bad — data might not have clear cluster structure

---

## 4.7 — Dimensionality Reduction for Visualization

### Problem: Can't Plot 14 Dimensions

Humans can see 2D (or maybe 3D). Our data has 14 dimensions. We need to project 14D → 2D while preserving as much structure as possible.

### PCA (Principal Component Analysis)

```
pca = PCA(n_components=2, random_state=RANDOM_SEED)
X_pca = pca.fit_transform(X_scaled)
var_exp = pca.explained_variance_ratio_
```

#### How PCA Works

PCA finds the **directions of maximum variance** in the data:

1. **Compute covariance matrix** (14×14 matrix showing how features co-vary)
2. **Find eigenvectors** — the principal components (PCs). PC1 captures the most variance, PC2 the second most, etc.
3. **Project data** onto the top 2 PCs

**Variance explained** tells you how much information is retained:

- `PC1 = 17.5%` → this single axis captures 17.5% of all variation in 14 features

- `PC2 = 14.0%` → this axis captures another 14%
- Total: 31.5% — we're seeing less than a third of the full picture

**Limitations:** PCA is **linear** — it can only find straight-line projections. Complex non-linear relationships between features are lost.

## t-SNE (t-distributed Stochastic Neighbor Embedding)

```
tsne = TSNE(n_components=2, perplexity=30, random_state=RANDOM_SEED, n_it
X_tsne = tsne.fit_transform(X_scaled[tsne_idx])
```

#### How t-SNE Works (Simplified)

1. In 14D space: compute probability that each pair of points are "neighbors" (using Gaussian distribution)
2. In 2D space: randomly place all points, compute neighbor probabilities (using t-distribution)
3. **Minimize the difference** between 14D and 2D neighbor probabilities using gradient descent
4. Points that are close in 14D get pulled together in 2D; distant points get pushed apart

#### PCA vs t-SNE

| Property | PCA | t-SNE |
|---|---|---|
| Method | Linear projection | Non-linear optimization |
| Preserves | Global structure (distances) | Local structure (neighborhoods) |
| Speed | Very fast | Slow ($O(N^2)$) |
| Deterministic | Yes | Partially (stochastic) |
| Distances meaningful? | Yes (PC1 axis = most variance) | No (distances are arbitrary) |
| Good for | Seeing overall spread | Seeing cluster separation |

`perplexity=30` : Roughly "how many neighbors to consider." Lower = more local detail. Higher = more global structure. 30 is a common default for datasets of 1,000–10,000 points.

`n_iter=1000` : Gradient descent iterations. More iterations = better convergence but slower. 1000 is usually sufficient.

## Sampling for Visualization

```
N_TSNE = min(5000, len(X_scaled))
tsne_idx = np.random.default_rng(RANDOM_SEED).choice(len(X_scaled), N_TSN
```

t-SNE is $O(N^2)$ in memory and time. With 3,600 points it's fine, but the code caps at 5,000 as a safeguard for larger datasets.

---

# Key Takeaways for Writing This From Scratch

1. **Always test multiple K values** — don't assume K=5 just because there are 5 roles
2. **Use two metrics** (silhouette + BIC) — they measure different things and consensus is more robust
3. **Label alignment is essential** — different algorithms produce arbitrary label numbers
4. **Ensemble > single algorithm** — KMeans alone misses non-spherical clusters
5. **GMM probabilities are gold** — they give confidence scores that no other algorithm provides
6. **t-SNE is for visualization only** — don't cluster on t-SNE coordinates
7. **PCA variance explained** tells you how much you're missing — 31% means 69% of information is lost in the plot

# Explanation 5: Role Assignment, Player Aggregation & Validation

**Covers:** Lines 756–1037 of `code_edited.py`

---

## 5.1 — Cluster Profiling: Understanding What Each Cluster "Looks Like"

### Mean Feature Table

```
ORIGINAL_COLS = ["damage_dealt", "kill_count", "utility_used", "distance_
                 "mean_isolation", "first_engagement_rel", "death_tick_re
                 "zone_primary", "headshot_pct", "trade_kills", "flash_as
                 "multi_kill_round", "survived", "opening_duel_won"]

cluster_profile = g2_df.groupby("cluster_id")[orig_cols_present].mean().r
```

This computes the **mean of each raw feature** per cluster. The result is a K×14 table:

```
            damage_dealt  kill_count  utility_used  mean_isolation  ...
cluster_id
0                  85.2         1.2           2.8           350.5     ...
1                 165.3         2.5           1.1           890.2     ...
2                  45.8         0.6           3.5           220.1     ...
3                 120.7         1.8           0.9          1200.8     ...
4                  95.0         1.5           2.2           510.3     ...
```

By reading this table, you can already guess roles:

- Cluster 1: high damage, high kills, high isolation → Lurker or Entry?
- Cluster 2: low kills, high utility, low isolation → Support
- Cluster 3: moderate kills, very high isolation → Lurker

### Normalised Heatmap

```
norm_profile = ((cluster_profile - cluster_profile.min()) /
                (cluster_profile.max() - cluster_profile.min() + 1e-9))
```

**Min-max normalization:** Maps each feature column to [0, 1] where:

- 0 = the cluster with the LOWEST mean for that feature
- 1 = the cluster with the HIGHEST mean

The `+ 1e-9` prevents division by zero if all clusters have the same value for a feature.

This is different from StandardScaler:

- StandardScaler: z-score across ALL data points (used for clustering input)
- Min-max here: normalises across K cluster MEANS (used for visualization only)

The heatmap uses the `RdYlGn` colormap (Red → Yellow → Green) with the actual raw values as annotations. So you see both the color (relative comparison) and the number (absolute value).

---

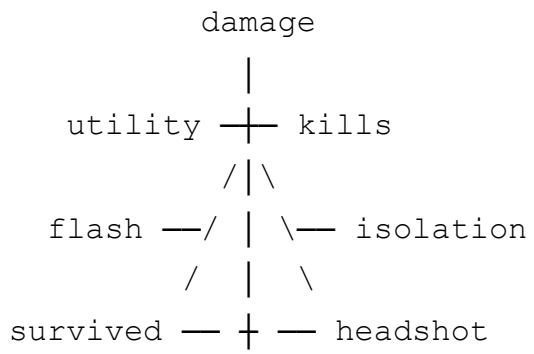
# 5.2 — Radar Charts (Spider Charts)

```
radar_cols = ["damage_dealt", "kill_count", "utility_used", "distance_tra
              "mean_isolation", "first_engagement_rel", "headshot_pct",
              "trade_kills", "flash_assists", "survived", "opening_duel_w

angles = np.linspace(0, 2 * np.pi, len(radar_present), endpoint=False).to
angles += angles[:1]   # Close the polygon
```

## How Radar Charts Work

A radar chart has one axis per feature, arranged radially around a circle. Each cluster's profile is plotted as a polygon connecting its normalised values on each axis.



```
                    |
first_engagement
```

### Why `angles += angles[:1]`?

To close the polygon, we need to connect the last axis back to the first. Adding the first angle at the end creates this wrap-around connection.

### Reading Radar Charts for CS2 Roles

**Ideal Entry Profile:**

```
High: damage, kills, opening_duel, multi_kill
Low:  first_engagement (early), survived (dies often)
Medium: everything else
```

**Ideal Lurker Profile:**

```
High: isolation, distance, first_engagement (late), survived
Low:  utility, trade_kills, flash_assists
```

**Ideal Support Profile:**

```
High: utility, trade_kills, flash_assists
Low:  isolation, multi_kill
```

When looking at the radar charts, you should be able to match each cluster's shape to one of these profiles.

---

## 5.3 — Role Assignment: The Weighted Scoring Matrix

This is the most domain-specific part of the pipeline. It translates cluster statistics into CS2 role names.

### Step 1: Compute Scaled Cluster Means

```
scaled_cols_present = [c for c in g2_df.columns if c.endswith("_scaled")]
cluster_scaled = g2_df.groupby("cluster_id")[scaled_cols_present].mean()
```

This computes the mean **scaled** feature value for each cluster. Scaled values (z-scores) are used instead of raw values so that all features contribute equally.

## Step 2: Column Name Lookup

```
def _find_col(cols, keyword):
    matches = [c for c in cols if keyword in c]
    return matches[0] if matches else None

fe_col   = _find_col(cluster_scaled.columns, "first_engagement")
dmg_col  = _find_col(cluster_scaled.columns, "damage")
...
```

**Why this lookup?** Column names after preprocessing are like `"damage_dealt_log_scaled"` or `"kill_count_scaled"`. Instead of hardcoding exact names (which would break if preprocessing changes), we search by keyword.

## Step 3: The Role Scoring Matrix

```
roles_def = {
    "Entry":   {fe_col: -2.5, dmg_col: +2.0, kill_col: +1.5,
                surv_col: -1.0, open_col: +2.0, multi_col: +1.0},
    "Support": {ut_col: +3.0, iso_col: -2.0, trade_col: +2.0,
                flash_col: +2.0, surv_col: +0.5},
    "Lurker":  {iso_col: +2.5, fe_col: +1.5, dist_col: +1.0,
                surv_col: +1.0, trade_col: -1.0},
    "AWPer":   {hs_col: -2.5, dmg_col: +1.5, dist_col: -1.5,
                kill_col: +1.0, iso_col: +0.5},
    "IGL":     {ut_col: +1.5, surv_col: +2.0, open_col: -1.5,
                flash_col: +1.0, kill_col: -1.0, multi_col: -1.0},
    "Rifler":  {dmg_col: +2.0, kill_col: +2.0, hs_col: +1.5,
                fe_col: -1.0, surv_col: +0.5, multi_col: +1.0},
}
```

#### How to Read This Matrix

Each role is defined by a set of **weighted expectations** on features:

- **Positive weight (+2.0)** means "this role should have HIGH values on this feature"
- **Negative weight (-2.5)** means "this role should have LOW values on this feature"
- **Absent feature** means "this feature doesn't matter for this role"

#### Entry Fragger Analysis

```
"Entry": {
    fe_col: -2.5,       # VERY LOW first_engagement (attacks EARLY)
    dmg_col: +2.0,      # HIGH damage
    kill_col: +1.5,     # HIGH kills
    surv_col: -1.0,     # LOW survival (dies often)
    open_col: +2.0,     # HIGH opening duel (takes first fights)
    multi_col: +1.0     # SOME multi-kills (star power)
}
```

**CS2 Context:** Entry fraggers (like apEX, ropz) are the first players to enter a bombsite. They:

- Engage enemies first (low `first_engagement_rel`)
- Deal heavy damage and get kills
- Die frequently (low `survived`) because they're first in and take enemy crossfire
- Win or lose the opening duel (high `opening_duel_won`)

**AWPer Analysis**

```
"AWPer": {
    hs_col: -2.5,       # VERY LOW headshot % (AWP kills are body shots)
    dmg_col: +1.5,      # HIGH damage (AWP does 100+ per shot)
    dist_col: -1.5,     # LOW distance (holds angles, doesn't move much)
    kill_col: +1.0,     # SOME kills
    iso_col: +0.5       # SLIGHTLY isolated (plays off-angles)
}
```

**CS2 Context:** The AWP is a sniper rifle that kills with one body shot. AWPers (like ZywOo, s1mple):

- Have **low headshot %** because the AWP doesn't need headshots — a body shot kills instantly
- Deal high damage per engagement
- Play **statically** — they hold long angles and wait for enemies to peek
- Are slightly isolated because sniper positions are often off the main path

**Rifler Analysis (FIX 4 — New in v3)**

```
"Rifler": {
    dmg_col: +2.0,      # HIGH damage
    kill_col: +2.0,     # HIGH kills
    hs_col: +1.5,       # HIGH headshot % (AK/M4 headshots)
    fe_col: -1.0,       # SOMEWHAT early engagement
    surv_col: +0.5,     # MODERATE survival
```

```
    multi_col: +1.0     # SOME multi-kills
}
```

**Why this role was added (FIX 4):** In v2, there were only 5 roles but often 5+ clusters. One cluster was typically a "generic rifler" — moderate on everything. Without a Rifler role, this cluster was force-assigned to AWPer (the least-bad fit), which made no sense because riflers have HIGH headshot %, while AWPers have LOW.

## Step 4: Computing Scores

```
scores = {cid: {role: 0.0 for role in roles_def} for cid in cluster_scale

for role, weights in roles_def.items():
    for col, w in weights.items():
        if col and col in cluster_scaled.columns:
            for cid in cluster_scaled.index:
                scores[cid][role] += w * float(cluster_scaled[col].loc[ci
```

For each cluster-role pair, the score is the **dot product** of the weight vector and the cluster's mean feature vector.

**Concrete Example**

Cluster 1 has these scaled means:

```
first_engagement_scaled = -0.8  (early engager)
damage_scaled           = +1.2  (high damage)
kill_count_scaled       = +0.9  (above average kills)
survived_scaled         = -0.5  (dies often)
opening_duel_scaled     = +0.6  (wins opening duels)
multi_kill_scaled       = +0.3  (some multi-kills)
```

Score for "Entry":

```
= (-2.5 × -0.8) + (2.0 × 1.2) + (1.5 × 0.9) + (-1.0 × -0.5) + (2.0 × 0.6)
+ (1.0 × 0.3)
= 2.0 + 2.4 + 1.35 + 0.5 + 1.2 + 0.3
= 7.75  ← HIGH SCORE → this cluster matches Entry well
```

Score for "IGL":

```
= (1.5 × utility_scaled) + (2.0 × -0.5) + (-1.5 × 0.6) + (1.0 ×
flash_scaled) + (-1.0 × 0.9) + (-1.0 × 0.3)
```

```
= ... some utility contribution ... + (-1.0) + (-0.9) + ... + (-0.9) +
(-0.3)
= probably negative → this cluster does NOT match IGL
```

### Step 5: Greedy Assignment (No Duplicate Roles)

```
assigned, used_roles, all_pairs = {}, set(), []
for cid, role_scores in scores.items():
    best_score = max(role_scores.values())
    all_pairs.append((best_score, cid, role_scores))
all_pairs.sort(reverse=True)   # Process highest-confidence clusters firs

for _, cid, role_scores in all_pairs:
    for role, _ in sorted(role_scores.items(), key=lambda x: -x[1]):
        if role not in used_roles:
            assigned[cid] = role
            used_roles.add(role)
            break
```

**The constraint:** Each role can only be assigned to ONE cluster. We don't want two clusters both labeled "Entry."

**The algorithm:**

1. Sort all clusters by their best score (most confident first)
2. For each cluster: try to assign its highest-scoring role
3. If that role is already taken → try the second-highest, then third, etc.
4. If ALL roles are taken → assign "Flex"

**Example with K=5:**

```
Cluster 3: best=Entry(8.2)    → Assigned: Entry ✅
Cluster 1: best=Entry(7.5)    → Entry taken! 2nd: Rifler(5.1) → Assigned:
Rifler ✅
Cluster 0: best=Support(6.0)  → Assigned: Support ✅
Cluster 4: best=Lurker(4.5)   → Assigned: Lurker ✅
Cluster 2: best=AWPer(3.8)    → Assigned: AWPer ✅
```

---

# 5.4 — Role Label Distribution

```
g2_labeled = g2_df.copy()
g2_labeled["role_label"] = g2_labeled["cluster_id"].map(role_map)
```

`role_map` is like `{0: "Support", 1: "Rifler", 2: "AWPer", 3: "Entry", 4: "Lurker"}`. The `.map()` translates every row's `cluster_id` to a role name.

### Role Distribution Visualization

```
role_colors = {
    "Entry": "#E63946",  "Support": "#2A9D8F",  "Lurker": "#E9C46A",
    "AWPer": "#457B9D",  "IGL":     "#8338EC",  "Rifler": "#F4A261",
    "Flex":  "#999999",
}
```

Each role gets a consistent color across all visualizations:

- **Entry: Red** — aggressive, dangerous
- **Support: Teal** — team-oriented, calm
- **Lurker: Gold** — sneaky, deceptive
- **AWPer: Steel Blue** — precision, cold
- **IGL: Purple** — leadership, strategy
- **Rifler: Orange** — versatile, warm
- **Flex: Gray** — undefined

---

## 5.5 — Player-Level Aggregation

### Why Aggregate?

Up to this point, every row is a **player-round** — ZywOo has ~360 rows (12 maps × 30 rounds). Each row has its own role label. But ZywOo might be labeled:

- Entry in 50 rounds
- AWPer in 250 rounds
- Rifler in 60 rounds

We want to assign ONE role per player.

### The Aggregation Logic

```
for player_name, pgroup in g2_labeled.groupby("player_name"):
    role_counts = pgroup["role_label"].value_counts()
    dominant_role = role_counts.index[0]        # Most frequent role
    consistency = role_counts.iloc[0] / len(pgroup)  # % of rounds in dom
    final_role = dominant_role if consistency >= 0.50 else "Flex"
```

**Step-by-step for "ZywOo":**

1. Filter all rows where `player_name == "ZywOo"` → 360 rows
2. Count role labels:
   ```
   AWPer:   250 (69.4%)
   Entry:    50 (13.9%)
   Rifler:   60 (16.7%)
   ```
3. `dominant_role = "AWPer"` (most frequent)
4. `consistency = 250/360 = 0.694` (69.4%)
5. `0.694 >= 0.50` → `final_role = "AWPer"` ✅

**For a flex player like "ropz":**

1. 300 rows with:
   ```
   Lurker:  130 (43.3%)
   Rifler:  100 (33.3%)
   Entry:    70 (23.3%)
   ```
2. `dominant_role = "Lurker"` (most frequent)
3. `consistency = 130/300 = 0.433` (43.3%)
4. `0.433 < 0.50` → `final_role = "Flex"` (not consistent enough)

## Why 50% Threshold?

If a player is in one role less than half the time, they're genuinely **flexible** — playing different roles depending on the round/situation. Labeling them as their most common role would be misleading.

## Additional Player Stats

```
mean_stats = pgroup[orig_cols_present].mean().to_dict()
mean_stats.update({
    "player_name": player_name,
    "dominant_role": dominant_role,
    "final_role": final_role,
    "role_consistency": round(consistency, 3),
```

```
        "total_rounds": len(pgroup),
        "total_matches": pgroup["match_id"].nunique(),
        "mean_gmm_confidence": round(pgroup["gmm_max_prob"].mean(), 3),
    })
```

Each player gets:

- **Average stats** across all their rounds (mean damage, mean kills, etc.)
- **Role consistency** — how reliably they play one role
- **GMM confidence** — how clearly their rounds cluster (higher = more distinct playstyle)

### Role Consistency Histogram

```
ax1.hist(player_roles_df["role_consistency"], bins=20)
ax1.axvline(0.50, color="#E63946", linestyle="--", label="Flex threshold
```

This shows the distribution of consistency scores:

- **Peak near 1.0** = many players have very consistent roles (good!)
- **Peak near 0.3** = many players are flexible (might need more data)
- **The red dashed line at 0.50** shows the Flex threshold

---

## 5.6 — Ground Truth Validation (Confusion Matrix)

```
if "true_role" in g2_labeled.columns:
    cm = confusion_matrix(true_labels, pred_labels, labels=unique_roles)
    accuracy = np.trace(cm) / cm.sum()
```

### When Is This Used?

Only when the data has a `true_role` column — which only exists in **synthetic data** (where we generated fake players with known roles). Real demo data doesn't have ground truth roles, so this section is skipped.

### How a Confusion Matrix Works

```
                Predicted
          Entry  Support  Lurker  AWPer  IGL
```

```
True   Entry     180       15        5        0      0     → 90% correct
       Support    10      175       10        3      2     → 87.5% correct
       Lurker      5        8      185        0      2     → 92.5% correct
       AWPer       0        2        0      190      8     → 95% correct
       IGL         5       10        3        7    175     → 87.5% correct
```

- **Diagonal** = correct predictions
- **Off-diagonal** = misclassifications
- **Accuracy** = sum(diagonal) / total = (180+175+185+190+175) / 1000 = 90.5%

The `np.trace(cm)` function sums the diagonal elements.

### Seaborn Heatmap Visualization

```
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=unique_roles, yticklabels=unique_roles)
```

- `annot=True` — shows numbers inside each cell
- `fmt="d"` — format as integers (not decimals)
- `cmap="Blues"` — darker blue = higher count

---

## 5.7 — Final Summary & Output Verification

```
expected_outputs = [
    "g1_features.parquet", "g1_clean.parquet",
    "g2_clustered.parquet", "g2_labeled.parquet", "g2_player_roles.parque
    "g2_correlation_matrix.png", "g2_k_selection.png",
    "g2_pca_tsne.png", "g2_profile_heatmap.png",
    "g2_radar_charts.png", "g2_roles_final.png", "g2_player_roles.png",
]
for f in expected_outputs:
    status = "✅" if (OUTPUT_DIR / f).exists() else "❌ MISSING"
```

This verifies that all expected output files were actually created. Any ❌ MISSING indicates a section that failed silently.

### Complete Output File Map

```
processed/
│
├── DATA FILES (Parquet format — columnar storage, fast read/write)
│   ├── g1_features.parquet       Raw 14-feature data (before scaling)
│   │                             Schema: match_id, player_name,
round_num, side, + 14 features
│   │                             ~3,600 rows
│   │
│   ├── g1_clean.parquet          Scaled data (after preprocessing)
│   │                             Same rows + 14 additional "_scaled"
columns
│   │
│   ├── g2_clustered.parquet      + cluster_id + gmm_max_prob columns
│   │                             Each row now knows its cluster
│   │
│   ├── g2_labeled.parquet        + role_label column
│   │                             Each row now has a role name
│   │
│   └── g2_player_roles.parquet   Player-level summary (~30 rows)
│                                 One row per player with final_role,
consistency, mean stats
│
├── VISUALIZATION FILES (PNG, 150 DPI)
│   ├── g2_correlation_matrix.png   14×14 feature correlation heatmap
│   ├── g2_k_selection.png          Silhouette + BIC vs K charts
│   ├── g2_pca_tsne.png             PCA and t-SNE cluster views (side by
side)
│   ├── g2_profile_heatmap.png      Cluster × feature mean heatmap
│   ├── g2_radar_charts.png         One radar chart per cluster
│   ├── g2_roles_final.png          Role distribution bar chart + PCA with
role colors
│   ├── g2_player_roles.png         Player consistency histogram + player
role distribution
│   └── g2_confusion_matrix.png     (Only if synthetic data) Predicted vs
true confusion matrix
```

## Why Parquet Format?

```
features_df.to_parquet(OUTPUT_DIR / "g1_features.parquet", index=False)
```

| Format | Size for ~3,600 rows | Read Speed | Features |
|---|---|---|---|
| CSV | ~2 MB | Slow (text parsing) | Human-readable |
| Parquet | ~0.5 MB | Very fast (columnar) | Type-preserving, compressed |
| Pickle | ~1 MB | Fast | Python-only, version-sensitive |

Parquet is the standard for data science workflows because:

1. **Column types are preserved** (int stays int, float stays float)
2. **Columnar storage** means reading only specific columns is fast
3. **Built-in compression** (Snappy by default)
4. **Language-neutral** — readable by Python, R, Spark, etc.

---

# 5.8 — The Complete Pipeline Flow (End-to-End)

```
12 .dem files (binary, ~1.5 GB total)
    │
    ▼  [parse_dem_file × 12]
    │
5 event DataFrames per demo (kills, damage, positions, grenades,
round_ends)
    │
    ▼  [reconstruct_rounds + assign_round_num]
    │
Events tagged with round_num
    │
    ▼  [extract_features_from_dem × 12]
    │
14-feature DataFrame (~3,600 rows × 18 cols)
    │
    ├──→ g1_features.parquet (raw)
    │
    ▼  [preprocess: clip → log1p → StandardScaler]
    │
Scaled DataFrame (~3,600 rows × 36 cols)
    │
    ├──→ g1_clean.parquet (scaled)
    │
    ▼  [KMeans + GMM + HDBSCAN → ensemble consensus]
    │
```

```
Clustered DataFrame (+ cluster_id, gmm_max_prob)
    │
    ├──→ g2_clustered.parquet
    │
    ▼  [score_clusters → greedy role assignment]
    │
Labeled DataFrame (+ role_label)
    │
    ├──→ g2_labeled.parquet
    │
    ▼  [groupby player_name → mode role → consistency check]
    │
Player-Level Summary DataFrame (~30 rows)
    │
    └──→ g2_player_roles.parquet
```

---

# Key Takeaways for Writing This From Scratch

1. **Cluster profiling comes BEFORE role assignment** — you need to understand what each cluster looks like before labeling it
2. **The scoring matrix encodes domain knowledge** — weights must reflect real CS2 role definitions
3. **Greedy assignment prevents duplicate roles** — but can be suboptimal (Hungarian algorithm would be better)
4. **Player-level aggregation is essential** — per-round roles are noisy, per-player roles are meaningful
5. **The 50% consistency threshold** prevents misleading labels — a player who plays 40% Lurker and 35% Entry shouldn't be called a Lurker
6. **GMM confidence scores** add another dimension — players with low gmm_max_prob are genuinely ambiguous in their playstyle
7. **Parquet > CSV** for intermediate data — preserves types, compresses well, reads fast
8. **Always verify outputs exist** — catching silent failures early saves debugging time